

Dataspace-POC: Evaluierung & Implementierungsplan

AM2Scale · Minimum Viable Dataspace (Eclipse EDC) · Stand: 7. Juli 2026

1. Use Case

Wir bauen eine Dataspace-Lösung, mit der Unternehmen Daten sicher und kontrolliert untereinander austauschen können – jedes Unternehmen kann dabei sowohl Anbieter als auch Empfänger von Daten sein. Ein Unternehmen stellt ein Datenprodukt bereit, legt fest, unter welchen Bedingungen es genutzt werden darf (z. B. Zweck, Laufzeit, Berechtigungen), macht es über einen Katalog auffindbar; ein anderes Unternehmen kann es finden, die Nutzungsbedingungen akzeptieren und die Daten sicher und nachvollziehbar beziehen – ohne unkontrollierte Kopien oder Zugriffe.

Langfristiges Ziel ist der Austausch von **3D-Druck-Dateien** zwischen Unternehmen: Ein Unternehmen stellt ein Bauteil-Design bereit, das ein Partnerunternehmen für die Fertigung nutzen darf – mit geregelter Zugriff, Nachvollziehbarkeit und Schutz des geistigen Eigentums. Die Dateien liegen in einem Speicher und sollen perspektivisch mit einem **Knowledge Graph** (Bauteil-, Material- und Prozess-Metadaten) verknüpft werden. Zusätzlich sollen Empfänger **Qualitätsdaten an den Anbieter zurückspielen** können – über denselben kontrollierten Weg.

2. Ist-Stand

- Lauffähiges MVD (Eclipse EDC 0.17) auf Kubernetes/Kind mit Podman: Provider, Consumer, Issuer, IdentityHub, Vault, Keycloak, Traefik.
- Beide Teilnehmer besitzen eine eigene Dataplane und können Assets anbieten *und* beziehen (bidirektional).
- Vollständige Protokoll-Flows als Bruno-Collections: Katalog → Contract Negotiation → Agreement → Transfer → EDR → Download, getrennt als „Consumer to Provider“ und „Provider to Consumer“.
- Bild-Asset (asset-3) als realistisches Beispiel-Datenprodukt eingebunden.
- Einfache lokale Consumer-UI, die den kompletten Use-Case-Durchlauf abbildet.

3. Gap-Analyse: Was für den POC fehlt

A) Persistenz – aktuell komplett flüchtig

Das Start-Skript löscht bei jedem Lauf den gesamten Cluster; Postgres läuft ohne Volume; Vault im Dev-Modus (in-memory). Nach jedem Neustart ist der gesamte Zustand (Assets, Verträge, Identitäten, Secrets) verloren. **Muss zuerst gelöst werden** – sonst verschwinden die Ergebnisse aller weiteren Features bei jedem Neustart.

B) Teilnehmer sind statisch einbetoniert

Provider und Consumer existieren als handgepflegte, nahezu identische Manifest-Ordner mit hartkodierten DIDs, Hostnamen und Seed-Jobs. Für N Unternehmen braucht es ein parametrisiertes Template plus Automatisierung der Identitätskette: Keypair erzeugen, DID/Participant-Context im IdentityHub anlegen, Membership-Credential vom Issuer ausstellen, Dataplane registrieren. Dieses dynamische Onboarding mit Credential-Ausstellung ist zugleich der wissenschaftlich interessanteste Teil (Dataspace-Governance).

C) Kein Teilnehmer-Verzeichnis

Die UI kennt genau einen hartkodierten Counterparty. Mit mehreren Unternehmen braucht es eine Registry („Wer ist im Dataspace?“), die beim Onboarding gepflegt wird und deren Kataloge die UI abfragt.

D) UI ist Single-Tenant und Single-Direction

Die bestehende UI bedient nur den Consumer und nur eine Richtung. Ziel: Login pro Unternehmen (Keycloak vorhanden), danach die eigene Sicht: fremde Kataloge durchsuchen, Protokoll durchlaufen, eigene Assets anlegen und verwalten.

E) Echte Dateien statt Demo-APIs

Assets zeigen aktuell auf Demo-Endpunkte. Für 3D-Druck-Dateien braucht jedes Unternehmen einen File-Store (Upload über UI, Auslieferung über die Dataplane), und heruntergeladene Dateien müssen persistent „beim Unternehmen“ gespeichert werden.

F) Qualitätsdaten-Rückfluss

Konzeptionell bereits vorhanden (bidirektionaler Austausch), muss generalisiert werden: Empfänger lädt Qualitätsbericht hoch → Asset mit Policy, die nur den ursprünglichen Anbieter (DID) zulässt → Anbieter bezieht ihn über denselben Protokollweg. Wichtiges Argument: Auch der Rückkanal läuft über die Dataspace-Governance, nicht per E-Mail.

G) Präsentierbarkeit fürs Paper

- **Negativ-Demo:** Unternehmen ohne gültiges Credential wird abgelehnt → beweist reale Policy-Durchsetzung.
- **Audit-Sicht:** Agreements und Transfer-Historie pro Unternehmen (Nachvollziehbarkeit als Kernargument).
- Sichtbarkeit der Protokoll-Zustände (Negotiation-States) in der UI.

4. Getroffene Architektur-Entscheidungen

Entscheidung	Festlegung & Begründung
Anzahl Unternehmen	4–5 , um alle Use-Case-Rollen abzudecken. Machbar auf Laptop (~10–12 GB RAM), unkritisch auf Server.

Fähigkeiten pro Unternehmen	Jedes Unternehmen erhält den vollen Stack (kann anbieten und beziehen). Die Konfiguration wird beim Anlegen festgelegt – keine Stack-Varianten, ein Template.
Topologie	Ein Connector-Stack pro Unternehmen (realitätsnah, 1:1-Abbildung). Zur Ressourcen-Schonung geteilte Infrastruktur: eine Postgres-Instanz (getrennte DBs) und ein Vault (getrennte Pfade) für alle Teilnehmer.
Orchestrierung	Migration von Kubernetes/Kind auf Docker Compose (Details Abschnitt 5): Ziel-Hosting ist Portainer, Persistenz über Volumes, ein Compose-Stack pro Unternehmen als Onboarding-Einheit.
Container-Engine	Podman bleibt. Compose ist ein engine-neutrales Format: lokal führt Podman die Stacks aus (Docker-kompatible API ist bereits aktiv), auf dem Server Docker oder Podman. Portainer und Traefik arbeiten über dieselbe API.
Login	Keycloak (bereits deployed): ein Realm-User pro Unternehmen, UI mappt Login auf den jeweiligen Connector.
File-Storage	Zunächst einfacher HTTP-File-Service pro Unternehmen (Dataplane unterstützt HttpData bereits). MinIO/S3 als optionale spätere Ausbaustufe.
Vault	Ein geteilter Vault mit File-Backend (persistent) statt Dev-Modus – näher am realen Betrieb, ohne Betriebskomplexität pro Firma.
Knowledge Graph	Metadaten-Schema für „3D-Druck-Asset“ (Bauteil, Material, Prozessparameter) wird früh definiert und in den Asset-Properties des Katalogs geführt; Anbindung eines Triple Store als spätere Ausbaustufe.
Hosting	Entwicklung lokal (Podman), Zielbetrieb auf Portainer-Server , zugänglich für Projektpartner.

5. Migration Kubernetes → Docker Compose

Warum: Portainer hostet nativ Compose-Stacks; Persistenz wird trivial (benannte Volumes); „ein Stack pro Unternehmen“ ist eine saubere Onboarding-Einheit (deploybar über die Portainer-API); die Kind/Podman-Fragilität auf Windows entfällt; Kollegen können das Setup ohne Kubernetes-Kenntnisse betreiben.

Warum das Eclipse EDC nicht gefährdet: EDC-Komponenten sind gewöhnliche Java-Anwendungen in Docker-Images ohne Kubernetes-Abhängigkeit. Das Dataspace-Protokoll (DSP, Negotiation, Transfer, EDR, DIDs, Verifiable Credentials) ist reines HTTP zwischen den Komponenten. Kubernetes ist im Upstream-MVD nur die Demo-Verpackung.

Kubernetes heute	Docker Compose morgen
Deployments	Services im Compose-File
ConfigMaps (configuration.properties)	gemountete Config-Dateien / Umgebungsvariablen
Cluster-DNS (*.svc.cluster.local)	Compose-Netzwerk + Service-Namen
Gateway API / HTTPRoutes / Ingress	Traefik mit Container-Labels (gleiche Middlewares)
Seed-Jobs	dieselben Skripte als One-Shot-Container
<code>kubect1 wait</code>	Healthchecks + <code>depends_on</code>

Aufwand & Einschränkungen: Die in DIDs und Configs eingebetteten k8s-Hostnamen müssen konsistent umgestellt werden – das ist im Wesentlichen dieselbe Parametrisierungsarbeit, die das dynamische Onboarding ohnehin erfordert (Synergie statt Doppelarbeit). Die mitgelieferten E2E-Tests bleiben zunächst k8s-gebunden; künftige Upstream-Merges werden aufwendiger (der Fork weicht bereits heute ab).

6. Implementierungsplan (Step by Step)

Phase	Inhalt	Ergebnis / Definition of Done
1 · Compose-Fundament	Infra-Stack (Traefik, Keycloak, Issuer, geteilter Postgres, Vault mit File-Backend) + parametrisiertes Company-Template; Provider und Consumer als erste zwei Unternehmen daraus instanziiert; Persistenz über benannte Volumes.	Bestehender Bruno-Flow läuft 1:1 gegen das Compose-Setup; Zustand überlebt Neustart von Stack und Rechner.
2 · Onboarding + Registry	Onboarding-Service mit API und UI-Formular: „Unternehmen anlegen“ (Name, Start-Konfiguration) → deployt Company-Stack, erzeugt Identität (Keypair, DID, Participant-Context), lässt Membership-Credential vom Issuer ausstellen, registriert Dataplane, trägt in Teilnehmer-Registry ein.	Ein neues Unternehmen ist ohne Handarbeit in wenigen Minuten voll teilnahmefähig und für andere im Dataspace sichtbar.
3 · Multi-Tenant-UI	Keycloak-Login pro Unternehmen; eigene Sicht: Kataloge aller anderen Teilnehmer (via Registry) durchsuchen, Protokoll-Durchlauf mit Statusanzeige, Download in firmeneigenen persistenten Speicher; eigene Assets per Datei-Upload anlegen (inkl. 3D-Druck-Metadaten-Schema, Policy, Contract Definition).	Jedes Unternehmen kann sich einloggen, Dateien anbieten und von anderen beziehen – vollständig über die UI, ohne Bruno.
4 · Qualitätsdaten-Rückfluss	Empfänger lädt Qualitätsbericht zum bezogenen Asset hoch → automatisch Asset mit DID-beschränkter Policy für den Ursprungs-Anbieter → dieser wird benachrichtigt bzw. sieht es im Katalog und bezieht es über den Standardweg.	Geschlossener Kreislauf Design-Datei → Fertigung → Qualitätsdaten, beidseitig in der UI sichtbar.

5 · Paper-Polish	Negativ-Demo (Ablehnung ohne Credential), Audit-/Agreement-Ansicht, Demo-Drehbuch für Präsentationen, Architekturdiagramm; optional: KG-Anbindung (Triple Store), MinIO/S3.	Vorführbarer, reproduzierbarer POC mit dokumentiertem Demo-Ablauf für das Paper.
------------------	---	--

Phase 1–2 bilden das technische Fundament; Phase 3 ist der größte Einzelblock, aber gut zerlegbar; Phase 5 richtet sich nach dem Paper-Zeitplan.

7. Risiken & offene Punkte

- **Ressourcen lokal:** 4–5 Unternehmen $\approx$ 15 EDC-Container + Infrastruktur; auf dem Laptop ggf. mit 2–3 Firmen entwickeln, Vollausbau auf dem Server.
- **DID/Hostname-Konsistenz:** Fehleranfälligster Teil der Migration; wird durch zentrales Templating (eine Quelle für alle Namen) entschärft.
- **Upstream-Divergenz:** Nach der Migration sind Upstream-MVD-Updates nur noch selektiv übernehmbar – bewusst akzeptiert.
- **Knowledge Graph:** Umfang der KG-Integration (nur Metadaten-Schema vs. echter Triple Store) nach Phase 3 anhand des Paper-Fokus entscheiden.
- **Portainer-Server:** Zugang, Ressourcen und Netzwerk-/DNS-Konzept (Hostnamen statt `*.localhost`) rechtzeitig vor dem Deployment klären.

Erstellt am 7. Juli 2026 · AM2Scale · Basis: Fork des Eclipse EDC Minimum Viable Dataspace (Release 0.17.0)